Tara complete safety reference

This document is a code-grounded reference of the safety chain inside Tara. It focuses on the actual implementation points that keep the agent bounded to an allowed lab or competition environment.

The goal is not to bypass the guardrails; it is to explain exactly how the project enforces them and how to restore or narrow the environment safely for research.

---

1. File map: where the safety system lives

These are the main implementation points:

- config/default.yaml — declared policy values for scope, egress, Tor, services, and offense/defense behavior
- main.py — bootstraps config, builds the context, installs the guard, and starts subsystems
- core/scope.py — the primary allowlist / deny-by-policy engine
- core/netguard.py — socket-level egress guard
- core/toolrunner.py — single chokepoint for every tool invocation
- defense/defense.py — attack detection and mitigation workflow
- defense/assurance.py — deduplication, scoping, confirmation, false-positive guard, validation
- defense/mitigation.py — reversible and policy-based mitigation rules
- offense/offense.py — review queue and safe flag handoff

---

2. High-level safety architecture

Tara implements a layered safety model:

1. Configuration defines the allowable environment.
2. ScopePolicy validates hostnames, IPs, CIDRs, ports, and filesystem roots.
3. NetworkGuard intercepts socket-level egress and blocks non-approved traffic.
4. ToolRunner is the lone entry for tools and validates targets before execution.
5. Defense logic scores findings and only acts when evidence is strong enough.
6. False positive guards and flag validation block garbage or unintended actions.

This is not a single switch; it is a chain of independent checks. If one layer misses something, the next one is intended to catch it.

3. Configuration safety values in default.yaml

From config/default.yaml:

```yaml
dry_run: false

scope:
  allowed_hosts: []
  allowed_cidrs: []
  allowed_ports: []
  allowed_paths:
    - "./state"
    - "./challenges"
  block_special: true
  resolve_check: true
  enforce_egress: true
  control_hosts: []
  allowed_onions: []

# Tor transport — only for allowed onions
# not used as a general bypass

tor:
  enabled: false
  socks_host: 127.0.0.1
  socks_port: 9050

llm:
  provider: anthropic
  model: claude-sonnet-4-5
  api_key_env: ANTHROPIC_API_KEY

agent:
  autonomy: auto
  max_iters: 25

flag_format: 'FLAG\{[A-Za-z0-9_]+\}'

services: []

defense:
  interval: 5.0
  auto_mitigate: false
  checker_allowlist: []

offense:
  interval: 10.0
  discover: true
  flag_file: null
  auto_submit: false

ui:
  enabled: false
  host: 127.0.0.1
  port: 8787
  token: null
```

What each field means

`dry_run`

- controls whether the tool is actually executed
- when true, the scope check still runs, but execution is skipped

`scope.allowed_hosts`

- explicit hostnames allowed by policy
- exact-match names are trusted in a narrow environment

`scope.allowed_cidrs`

- competition or lab network ranges
- if empty, every network operation is denied by policy until configured

`scope.allowed_ports`

- allowed ports on in-scope systems
- empty means any port if the host is allowed, unless additional checks are configured

`scope.allowed_paths`

- local filesystem roots permitted for reads/writes

`scope.block_special`

- denies link-local, metadata, multicast, CGNAT, and reserved ranges

`scope.resolve_check`

- resolves hostname to all IPs and validates each one
- mitigates DNS rebinding / name-based bypass

`scope.enforce_egress`

- installs the socket-level network guard

`scope.control_hosts`

- allows specific control-plane hosts like LLM endpoints or proxies

`scope.allowed_onions`

- explicit `.onion` allowlist, if Tor is used

`tor.enabled`

- Tor is off by default
- this prevents unrestricted onion probing without explicit policy

`defense.auto_mitigate`

- must be enabled intentionally before mitigation is installed

`offense.auto_submit`

- must be enabled intentionally before flags are automatically submitted

---

4. Core policy: ScopePolicy

The main logic is in core/scope.py. This is the central allowlist engine.

4.1 Safety design comments

The file begins with a clear description of the intent:

```
"""
Scope enforcement — the hard gate.

Every network-touching action in the agent must pass through a ScopePolicy
check before it runs. The policy is deny-by-default: a target is only reachable
if it is explicitly inside the configured competition scope.

This exists for two reasons that point the same way:
  1. Correctness — an attack-defense agent must only touch competition targets.
  2. Containment — a misfire that leaves the sandbox is how teams get
     disqualified, investigated, or fined. The gate makes that physically hard.
"""
```

This means the project intentionally treats scope as a hard boundary, not a convenience.

4.2 Special ranges denied by default

```
_SPECIAL_DENY = [
    ipaddress.ip_network("169.254.0.0/16"),   # link-local incl. cloud metadata
    ipaddress.ip_network("fe80::/10"),         # IPv6 link-local
    ipaddress.ip_network("100.64.0.0/10"),     # CGNAT
    ipaddress.ip_network("224.0.0.0/4"),       # multicast
    ipaddress.ip_network("240.0.0.0/4"),       # reserved
    ipaddress.ip_network("ff00::/8"),          # IPv6 multicast
]
```

This is a direct safety rule. It prevents accidental traffic to ranges that are almost never legitimate challenge targets.

4.3 Host allowlist logic

```
def _host_allowed(self, host: str) -> bool:
    if host in self.allowed_hosts:
        return True
    try:
        ip = normalize_ip(ipaddress.ip_address(host))
    except ValueError:
        return False
    if self.block_special and self._is_special(ip):
        return False
    return self._ip_in_cidrs(ip)
```

Explanation: - explicit hosts are accepted - raw IP literals are checked - dangerous ranges are rejected - otherwise the host must match a configured CIDR

4.4 Port validation

```
def _check_port(self, host, port) -> None:
    if port is not None and self.allowed_ports and int(port) not in self.allowed_
ports:
        raise ScopeViolation(
            f"port {port} on {host!r} is outside allowed ports "
            f"{sorted(self.allowed_ports)}"
        )
```

This reduces the chance of a mis-scoped tool reaching a dangerous port or unapproved service.

4.5 DNS resolution validation

```
def resolve_and_check(self, host: str, port: int | None = None) -> list[str]:
    self._check_port(host, port)
    try:
        ipaddress.ip_address(host)
        is_literal = True
    except ValueError:
        is_literal = False

    if is_literal:
        self.check_target(host, port)
        return [host]

    if host not in self.allowed_hosts and not self.resolve_check:
        raise ScopeViolation(f"hostname {host!r} not in allowed_hosts")

    try:
        infos = socket.getaddrinfo(host, port)
    except socket.gaierror as e:
        raise ScopeViolation(f"cannot resolve {host!r}: {e}")

    resolved = sorted({info[4][0] for info in infos})
    if not resolved:
        raise ScopeViolation(f"{host!r} resolved to nothing")

    approved = []
    for addr in resolved:
        ip = normalize_ip(ipaddress.ip_address(addr))
        explicit = host in self.allowed_hosts
        if self.block_special and self._is_special(ip) and not explicit:
            raise ScopeViolation(
                f"{host!r} resolves to special/metadata address {addr} —
refusing"
            )
        if explicit or self._ip_in_cidrs(ip):
            approved.append(addr)
        else:
            raise ScopeViolation(
                f"{host!r} resolves to {addr}, which is OUTSIDE scope "
                f"(possible DNS rebinding) — refusing"
            )
    log.debug("scope OK (resolved): %s -> %s", host, approved)
    return approved
```

This is one of the most important protections in the project. It prevents a hostname from being treated as safe just because it looks okay at the string level.

4.6 Onion validation

```
def valid_onion(addr: str) -> bool:
    name = addr.lower().split(":")[0]
    if not name.endswith(".onion"):
        return False
    label = name[:-len(".onion")]
    return bool(re.fullmatch(r"[a-z2-7]{16}|[a-z2-7]{56}", label))
```

```
def check_onion(self, host: str, port: int | None = None) -> None:
    name = host.lower().split(":")[0]
    if not valid_onion(name):
        raise ScopeViolation(f"{host!r} is not a well-formed onion address")
    if name not in self.allowed_onions:
        raise ScopeViolation(
            f"onion {name!r} is outside competition scope — refusing. Tor egress
"
            f"is restricted to scope.allowed_onions"
        )
    self._check_port(host, port)
```

This ensures onions are treated as a special restricted class, not a generic network destination.

4.7 Path validation

```
def check_path(self, path: str) -> None:
    import os
    rp = os.path.realpath(path)
    for root in self.allowed_paths:
        rr = os.path.realpath(root)
        if rp == rr or rp.startswith(rr + os.sep):
            return
    raise ScopeViolation(f"path {path!r} is outside allowed roots {self.allowed_p
aths}")
```

This is a sandbox boundary for local file access.

---

5. Socket-level guard: NetworkGuard

The second safety layer is in core/netguard.py.

5.1 Why this exists

The file explains the design intent:

```
"""
NetworkGuard — socket-level egress enforcement (defense in depth).

The ScopePolicy gate in the ToolRunner is advisory: it only sees targets a tool
declares. This guard sits UNDER that, at the socket layer, so that every single
outbound TCP connection the process makes is checked against scope — whether a
tool declared it, forgot to, or a third-party library opened it.
"""
```

That is the key idea: even if a tool misreports what it touches, the socket connection is still checked.

5.2 Default control-plane allowlist

```
def _seed_defaults(self) -> None:
    for h in ("127.0.0.1", "::1", "localhost"):
        self.control_hosts.add(h)
    for var in ("HTTPS_PROXY", "https_proxy", "HTTP_PROXY", "http_proxy"):
        val = os.environ.get(var)
        if val:
            host = urlparse(val if "://" in val else "http://" + val).hostname
            if host:
                self.control_hosts.add(host)
```

Loopback is allowed for local services. Proxy hosts are also allowed so the system can reach its configured control-plane endpoints.

5.3 The actual socket patch

```
def install(self) -> None:
    if _INSTALLED["on"]:
        return
    guard = self

    def guarded_connect(self_sock, address):
        try:
            host = address[0]
            port = address[1] if len(address) > 1 else None
            guard.authorize_ip(host, port)
        except ScopeViolation as v:
            log.warning("EGRESS BLOCKED %s: %s", address, v)
            raise OSError(f"egress blocked by scope guard: {v}")
        except Exception:
            log.warning("EGRESS BLOCKED (unparseable) %s", address)
            raise OSError("egress blocked by scope guard: unparseable address")
        return _ORIG_CONNECT(self_sock, address)
```

This is a direct, process-wide egress monitor. It denies any connection outside the control plane or approved competition scope.

5.4 getaddrinfo guard

```
def guarded_getaddrinfo(host, *a, **k):
    infos = _ORIG_GETADDRINFO(host, *a, **k)
    kept = []
    for info in infos:
        addr = info[4][0]
        try:
            guard.authorize_ip(addr, info[4][1] if len(info[4]) > 1 else None)
            kept.append(info)
        except ScopeViolation:
            continue
    if not kept:
        raise socket.gaierror(f"all addresses for {host!r} blocked by scope
guard")
    return kept
```

This defends against DNS-based surprises and prevents resolution to blocked addresses.

5.5 Authorization check

```
def authorize_ip(self, addr: str, port: int | None) -> None:
    if self._ip_is_control(addr):
        return
    try:
        ip = normalize_ip(ipaddress.ip_address(addr))
    except ValueError:
        raise ScopeViolation(f"egress to non-IP {addr!r} blocked")
    if self.policy.block_special and self.policy._is_special(ip):
        raise ScopeViolation(f"egress to special address {addr} blocked")
    if self.policy._ip_in_cidrs(ip) or str(ip) in self.policy.allowed_hosts:
        self.policy._check_port(addr, port)
        return
    raise ScopeViolation(f"egress to {addr}:{port} blocked — not control-plane
and not in scope")
```

This is the final hard stop at the OS socket layer.

---

6. Tool execution choke point: ToolRunner

The actual execution layer is in core/toolrunner.py.

6.1 Why this matters

The file explains:

```
"""
ToolRunner — the single chokepoint through which every tool call passes.

Responsibilities, in order:
  1. Look up the tool.
  2. Ask it what it will touch, and run those targets through the ScopePolicy
     (resolving names when enabled). Out of scope -> refused here, never runs.
  3. In dry-run mode, log the intended call and return without side effects.
  4. Execute under a hard wall-clock timeout so one stuck tool can't freeze the
     agent during a scored tick.
"""
```

This is the architectural key: there is exactly one path for tool execution.

6.2 Target validation before tool call

```
try:
    targets = tool.targets(**arguments)
    if targets:
        self.scope.check_targets(targets)
except ScopeViolation as v:
    log.warning("REFUSED %s: %s", name, v)
    return ToolResult(ok=False, output=f"scope refused: {v}")
```

This uses the tool's declared targets and validates them against the policy.

6.3 Dry-run safety mode

```
if self.scope.dry_run and targets:
    msg = f"[dry-run] would run {name} against {targets} with {arguments}"
    log.info(msg)
    return ToolResult(ok=True, output=msg, meta={"dry_run": True})
```

Even in dry-run mode, the safety checks still occur. The execution is prevented, but the intent is still validated.

6.4 Timeout safety

```
future = self._pool.submit(self._safe_run, tool, arguments)
try:
    result = future.result(timeout=tmo)
except FutTimeout:
    log.warning("tool %s timed out after %ss", name, tmo)
    result = ToolResult(ok=False, output=f"tool timed out after {tmo}s", meta={"t
imeout": True})
```

This prevents one slow or hung tool from blocking the whole loop.

6.5 Output cap

```
MAX_OUTPUT = 100_000
...
if result.output and len(result.output) > MAX_OUTPUT:
    result.output = result.output[:MAX_OUTPUT] + f"\n...[truncated {len(result.ou
tput) - MAX_OUTPUT} chars]"
```

This is not directly a security boundary, but it makes output bounded and predictable.

---

7. Defense logic and false-positive guardrails

The detection and action flow is in defense/defense.py and defense/assurance.py.

7.1 The threat model the code anticipates

The assurance file says:

```
"""
Assurance — defense against false positives, false alarms, and hallucinations.

Detection that acts on a single regex hit is dangerous in A/D: a false block can
drop legitimate traffic or the SLA checker and cost you the game, and a
hallucinated flag or patch wastes a tick.
"""
```

This is important because Tara is designed to avoid acting on weak or noisy evidence.

7.2 Deduplication

```
class AlertDeduper:
    def fresh(self, finding) -> bool:
        now = time.time()
        k = self._key(finding)
        last = self._seen.get(k)
        self._seen[k] = now
        return last is None or (now - last) > self.window
```

This avoids one repeated pattern causing a flood of duplicate findings.

7.3 Confidence scoring

```
class ConfidenceScorer:
    def score(self, findings: list) -> float:
        if not findings:
            return 0.0
        base = max(_SEV_BASE.get(f.severity, 0.1) for f in findings)
        distinct_rules = len({f.rule for f in findings})
        if distinct_rules >= 2:
            base += 0.20
        if any(str(f.rule).startswith("suricata:") for f in findings):
            base += 0.25
        if any(f.rule == "flag_out" for f in findings):
            base += 0.30
        return min(1.0, round(base, 3))
```

This requires multiple signals or direct evidence before the system shares or acts.

7.4 Confirmation before action

```
class Confirmer:
    def confirm(self, finding):
        if self.replay_fn is None:
            return None
        flow = getattr(finding, "flow", None)
        req = getattr(flow, "request", b"") if flow else b""
        if not req:
            return None
        resp = self.replay_fn(req)
        text = resp.decode("latin-1", "replace") if isinstance(resp, bytes) else
str(resp)
        if self.flag_re and self.flag_re.search(text):
            return Confirmation(True, "replay leaked a flag-formatted token")
        return Confirmation(False, "replay produced nothing anomalous")
```

This is the “detect → confirm → act” pattern. It lowers the chance of acting on a noisy or unverified signal.

7.5 False positive guard on block rules

```
class FalsePositiveGuard:
    def is_safe(self, rule) -> tuple[bool, str]:
        if rule.kind == "payload":
            literal = re.sub(r"\\(.)", r"\1", rule.pattern)
            if len(literal) < self.min_len:
                return False, f"pattern too short/generic ({literal!r})"
            try:
                rx = re.compile(rule.pattern)
            except re.error as e:
                return False, f"invalid regex: {e}"
            for sample in self.benign:
                if rx.search(sample):
                    return False, f"would block benign traffic: {sample[:40]!r}"
            for cp in self.checker:
                if rx.search(cp) or (cp in literal):
                    return False, "would block the SLA checker"
            return True, "ok"
```

This is a deliberate protection against harmful firewall rules. It ensures a mitigation does not accidentally block the checker or allowed traffic.

7.6 Flag validation

```
def validate_flag(token: str, flag_format: str | None) -> bool:
    if not token:
        return False
    if not flag_format:
        return True
    return re.fullmatch(flag_format, token.strip()) is not None
```

This prevents garbage strings or hallucinated tokens from being accepted as valid flags.

---

8. Mitigation rules: reversible and policy-bound

The mitigation system is in defense/mitigation.py.

8.1 Rule object

```
@dataclass
class BlockRule:
    rule_id: str
    kind: str
    pattern: str
    reason: str
    service: str = ""
    created: float = field(default_factory=time.time)
    active: bool = True
```

This is structured data, not ad hoc shell snippets. Each rule is explicit and reversible.

8.2 Payload vs source block

```
if matched and len(matched) >= 4:
    return BlockRule(
        rule_id=f"r{self._seq}", kind="payload",
        pattern=re.escape(matched), reason=f"{finding.rule}: {finding.reason}",
        service=getattr(flow, "service", "") if flow else "",
    )
```

The code prefers a precise payload rule over a broad source block. That limits impact and increases safety.

8.3 Emit-only firewall mode

```
class NftEmitter(Enforcer):
    def __init__(self, run: bool = False):
        self.run = run

    def apply(self, rule: BlockRule):
        cmd = self._cmd(rule)
        self.emitted.append(cmd)
        if self.run:
            subprocess.run(cmd, shell=True)
        else:
            log.info("nft (emit-only): %s", cmd)
```

This prevents the system from silently running OS firewall commands without explicit approval. It is safer than direct enforcement.

---

9. Offense review path

The safe flag flow is in offense/offense.py.

9.1 Auto-submit is opt-in

```
def _on_flag(self, flag, source, meta) -> str:
    if self.auto_submit:
        return self.sink.offer(flag, source=source, meta=meta).status
    return self.review.enqueue(flag, source=source, meta=meta)
```

This is a direct safeguard: flags are queued for review unless the operator explicitly turns on auto-submission.

9.2 Human review queue

The system keeps captured flags in a review state instead of submitting them immediately, which reduces mistaken submissions.

---

10. Startup flow and safety integration

The startup path in main.py wires everything together:

```
def _install_guard(cfg, scope):
    if not cfg.get("scope", {}).get("enforce_egress", True):
        return
    guard = NetworkGuard(scope, control_hosts=cfg.get("scope", {}).get("control_h
osts", []))
    llm = cfg.get("llm", {})
    base_url = llm.get("base_url")
    host = urlparse(base_url).hostname if base_url else _PROVIDER_HOSTS.get(
        llm.get("provider", "anthropic").lower())
    if host:
        guard.add_control_host(host)
    tor = cfg.get("tor", {})
    if tor.get("enabled"):
        guard.add_control_host(tor.get("socks_host", "127.0.0.1"))
    guard.install()
```

This is the actual installation of the socket guard. It only happens if egress enforcement is enabled.

Then the context is created with the same policy and runner:

```
def build_context(cfg: dict) -> AgentContext:
    scope = ScopePolicy.from_config(cfg)
    state = StateStore(cfg.get("state_dir", "./state"))
    _install_guard(cfg, scope)
    provider = build_provider(cfg)
    registry = _build_tools(cfg)
    runner = ToolRunner(registry, scope, state)
```

This demonstrates the entire safety chain in one place: config → scope → guard → runner → tools.

---

11. Alternative safe implementation patterns

The project’s current implementation is one way to do the safety chain. Other valid implementations would preserve the same goal — bounded execution with explicit allowlisting — while changing the shape of the code. The key idea is that the design must still fail closed, not open.

11.1 Alternative A: policy-as-objects with per-tool scopes

Instead of a single `ScopePolicy` object, a team could define a `PolicyBundle` and bind a policy to each tool or tool family.

```
from dataclasses import dataclass, field

@dataclass
class ToolPolicy:
    allowed_hosts: set[str] = field(default_factory=set)
    allowed_cidrs: list[str] = field(default_factory=list)
    allowed_ports: set[int] = field(default_factory=set)
    allow_dns_resolution: bool = True
    dry_run: bool = False

class SafeTcpProbe:
    policy = ToolPolicy(
        allowed_cidrs=["10.50.0.0/16"],
        allowed_ports={80, 443, 9000},
    )

    def run(self, host, port):
        # validate against self.policy before use
        pass
```

This is functionally similar to the current design: each tool has a narrow contract and the runner still applies the permission check before execution.

What changes: - the policy is no longer a single global object only - permissions are more explicit per tool family - safety is still enforced before network operations

Tradeoff: - more setup complexity - easier to forget to attach a policy at new tool-registration time

Why it is still safe: - every call still crosses a validation gate before the network action runs

11.2 Alternative B: firewall-backed enforcement

Instead of monkey-patching socket calls directly, the project could emit or enforce firewall rules at the OS layer. This is conceptually similar to `NftEmitter` in defense/mitigation.py.

```
class FirewallGuard:
    def __init__(self, allowed_cidrs):
        self.allowed_cidrs = allowed_cidrs

    def build_rules(self):
        return [
            "nft add table inet tara_scope",
            "nft add chain inet tara_scope output { type filter hook output
priority 0; }",
        ]

    def apply(self):
        for rule in self.build_rules():
            subprocess.run(rule, shell=True, check=True)
```

This reduces the chance that a user-space bug accidentally escapes the system, because the OS can enforce network destination restrictions itself.

What changes: - enforcement moves closer to the kernel - rule generation becomes a policy artifact

Tradeoff: - more platform-specific behavior - harder to test on non-Linux systems

Why it is still safe: - the OS-level rules are still allowlist-based and must be explicitly generated from the trusted policy set

11.3 Alternative C: proxy-and-bridge model

A safer deployment could route all traffic through a dedicated proxy or bridge that validates the destination before the request reaches the network.

```
class ScopedProxy:
    def __init__(self, policy):
        self.policy = policy

    def send(self, host, port, payload):
        if not self.policy.is_allowed(host, port):
            raise PermissionError(f"blocked by proxy: {host}:{port}")
        return self._forward(host, port, payload)
```

This gives a single, inspectable choke point in front of all outbound traffic.

What changes: - network calls are mediated by a dedicated proxy service - tool code is simpler because it does not handle raw sockets directly

Tradeoff: - requires infrastructure and operational upkeep - adds latency and complexity

Why it is still safe: - the proxy itself is the enforcement boundary and has no “open to all” path

11.4 Alternative D: service-specific policy object

Another valid alternative is to scope by service-specific config instead of by global network policy.

```
@dataclass
class ServicePolicy:
    name: str
    host: str
    port: int
    allowed_methods: set[str] = field(default_factory=lambda: {"GET", "POST"})
    allowed_paths: set[str] = field(default_factory=set)

    def allows(self, method: str, path: str) -> bool:
        return method in self.allowed_methods and (not self.allowed_paths or
path in self.allowed_paths)
```

This is useful when the agent is operating within a known service matrix. It works well for challenge lab experiments, where the service map is known in advance.

What changes: - the policy is more contextual and service-aware - the app checks both IP/port and HTTP path semantics

Tradeoff: - more code complexity - more explicit service mapping required

Why it is still safe: - it is still an allowlist model and still fails closed for any unsupported path or host

11.5 Alternative E: layered policy + event-callback validation

A more explicit architecture is to validate the target, then validate the action type, then validate the actual call at execution time. This makes the safety chain easy to audit and extend.

```
class SafeAction:
    def __init__(self, policy, action, target):
        self.policy = policy
        self.action = action
        self.target = target

    def validate(self):
        if not self.policy.is_allowed(self.target.host, self.target.port):
            raise PermissionError("invalid target")
        if self.action.kind not in self.policy.allowed_actions:
            raise PermissionError("invalid action kind")
```

This pattern keeps the decision tree explicit: - allowed target? - allowed action? - allowed path? - safe execution context?

Tradeoff: - a larger validation layer - more ceremony around each tool call

Why it is still safe: - validation is not optional and is performed before execution

11.6 What all of these alternatives share

Every alternative must keep the same core properties:

- explicit allowlisting rather than implicit trust
- local policy object or policy bundle
- a single choke point before execution
- boundary checks at the app, socket, and/or OS level
- deny or fail-closed on unmapped cases
- review, scoring, and false-positive guards before high-risk action

The project’s current implementation is one strong version of this pattern, but it is not the only way to achieve the same security posture.

---

12. Research and restore model

For a lab or research environment, the correct practice is:

1. keep the original default config as backup
2. create a narrower config that only covers the lab subnet
3. run with `enforce_egress: true`
4. keep `resolve_check` and `block_special` enabled
5. restore the original default config after the run

This preserves the integrity of the safety model while allowing bounded experimentation.

---

12. Safety conclusion

Tara’s design is intentionally layered:

- explicit allowlist of scope
- special-range blocklist
- DNS/IP validation
- socket-level egress enforcement
- tool execution choke point
- confirmation before action
- false positive vetoes
- restrictive flag submission

- reversible mitigation

The project treats containment as a core requirement, not an optional toggle. That is the central idea behind the safety chain.

---

13. Reference links

- config/default.yaml
- main.py
- core/scope.py
- core/netguard.py
- core/toolrunner.py
- defense/defense.py
- defense/assurance.py
- defense/mitigation.py
- offense/offense.py